

Authentication

1. Feature Overview

Staff authentication via username/password, JWT bearer tokens, profile retrieval, and password change. Roles loaded from DB on each request.

Status: Implemented

2. Business Purpose

Protect banking APIs and the Angular staff portal. Support lockout after failed logins and role-based access for customers, accounts, and employees.

3. User Workflow

1. Open `/` login page
2. Enter credentials (optional remember-me)
3. Receive JWT; land on `/app/dashboard`
4. Profile at `/app/profile`; change password at `/app/profile/password`
5. 401 from API → interceptor clears session → login

4. Execution Flow

See [CALL_FLOW.md](#) and [SEQUENCE_DIAGRAMS.md](#).

5. Database Tables

`users`, `roles`, `user_roles` (lock/failed-attempt fields on `users`).

6. REST APIs

Method	Path	Auth
POST	<code>/auth/login</code>	Public
GET	<code>/auth/me</code>	Authenticated
PUT	<code>/auth/password</code>	Authenticated

7. Controllers

- `com.bankone.auth.controller.AuthenticationController`
- `com.bankone.auth.controller.TestController` (GET `/api/hello`)

8. Services

- `AuthenticationService` — `login`, `getCurrentUserProfile`, `changePassword`

- `JwtService` — generate / validate / extract
- `LoginAttemptService` — `incrementFailedAttempts`
- `CustomUserDetailsService` — `loadUserByUsername`

9. Repositories

- `UserRepository`, `UserRoleRepository` (user package)
- `RoleRepository` (role package)

10. DTOs

`LoginRequest`, `LoginResponse`, `UserProfileResponse`, `ChangePasswordRequest`

11. Entities

`User`, `Role`, `UserRole` (+ `BankUserDetails` adapter)

12. Utility Classes

None dedicated; JWT via JJWT library.

13. Configuration Classes

- `com.bankone.common.config.SecurityConfig`
- `jwt.secret`, `jwt.expiration` in `application.properties`
- Frontend: `api-config.ts`, `auth.interceptor.ts`, `auth-guard.ts`

14. Validation Rules

- Change password: `@NotBlank`, new password `@Size(min=8)`, confirm must match (service-level)

15. Security Rules

- Stateless sessions; CSRF disabled
- Only `/auth/login` (and `OPTIONS/error`) `permitAll` among auth routes
- Roles not in JWT claims — refreshed from DB per request

16. Exception Handling

`JwtAuthenticationEntryPoint` (401), `JwtAccessDeniedHandler` (403), plus `GlobalExceptionHandler` for business errors.

17. Logging

Spring Security TRACE enabled in dev properties (noise in production).

18. Audit Events

No dedicated audit event table. User `lastLogin` / failed attempts updated on login path.

19. Testing Strategy

- Redeploy script smoke: `POST /auth/login` with `admin` / `Admin@123`
- Manual: wrong password → lockout behavior
- Guard: visit ADMIN route as EMPLOYEE

20. Future Extension Guide

- Embed roles in JWT (trade-off: stale roles vs DB hit)
- Refresh tokens
- MFA
- Move secrets to env / Liberty jasypt

Future Modification Guide

Requirement: Change JWT expiry

Item	Detail
Files	<code>BankOne/src/main/resources/application.properties</code>
Classes	<code>JwtService</code> (reads <code>@Value</code>)
Methods	<code>generateToken()</code> uses <code>jwt.expiration</code>
Impact	Active tokens keep old expiry until re-login; document for ops

Requirement: Change lockout threshold

Item	Detail
Files	<code>LoginAttemptService.java</code> , <code>User</code> entity fields, possibly <code>AuthenticationService.login()</code>
Methods	<code>incrementFailedAttempts()</code> , lock check in <code>BankUserDetails</code> / login
Impact	UX messaging on login page

Requirement: Add public endpoint

Item	Detail
Files	<code>SecurityConfig.java</code>
Methods	<code>securityFilterChain</code> → <code>authorizeHttpRequests</code>
Impact	Must not accidentally open <code>/accounts/**</code>

Call hierarchy (login)

```
AuthenticationController.login()  
    ↓  
AuthenticationService.login()  
    ↓  
AuthenticationManager.authenticate()  
    ↓  
CustomUserDetailsService.loadUserByUsername()  
    ↓  
JwtService.generateToken()
```